

UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

FACULTAD DE INGENIERÍA

DIVISIÓN DE INGENIERÍA ELÉCTRICA

INGENIERÍA EN COMPUTACIÓN



EXAMEN EXTRAORDINARIO

CONTROL INALÁMBRICO DE LÁMPARA INCANDESCENTE VÍA APP ANDROID

NOMBRE: JIMÉNEZ CHÁVEZ DANIEL ORLANDO

MATERIA: FUNDAMENTOS DE SISTEMAS EMBEBIDOS

CLAVE MATERIA: 1858

PERÍODO: 2

GRUPO: 51

SEMESTRE 2027-1

PROFESOR: DR. KEVIN ANDRAE DELGADO VARGAS

FECHA: 29 DE OCTUBRE DE 2026

REPOSITORIO: https://github.com/DanielJC1/fse-dimmer-wifi

VIDEO: **https://youtu.be/_CcwWs9EYxs**

1 Definición del sistema

Este proyecto consiste en el diseño y desarrollo de un sistema embebido que regula de forma continua la potencia entregada a una lámpara incandescente de 127V_{RMS} a 60Hz, es decir, regular su intensidad luminosa en un rango de 0 a 100% utilizando una aplicación móvil en Android de manera inalámbrica vía WiFi y funcionando de manera autónoma al encenderse.

El sistema utiliza una tarjeta de desarrollo programable Arduino UNO R4 Wifi que opera como punto de acceso (AP) y servidor HTTP. El Arduino recibe el porcentaje desde una aplicación en Android y controla el ángulo de disparo de un TRIAC mediante un módulo dimmer comercial de AC JTDB-V2.0.

Un sistema embebido se puede definir como una combinación de un sistema de cómputo, software y hardware diseñado para realizar una o algunas tareas específicas bajo restricciones de recursos.

Dado lo anterior, se confirma que este proyecto es un sistema embebido ya que ejecuta una función dedicada al regular la intensidad luminosa, se sincroniza en tiempo real con la red eléctrica, opera con los recursos limitados del Arduino (256 KB de memoria flash, 32 KB de SRAM) y funciona de forma autónoma, alimentado por una powerbank o cargador tipo USB-C de 5 V.

Requisito	Verificación	Resultado
Regular la potencia de 0 a 100 % de forma continua	Barrido del slider con observación visual del filamento.	Cumple con un brillo apreciable entre 11 % y 90 %
Control inalámbrico sin infraestructura externa	Conexión del teléfono al AP 192.168.4.1	Cumple
Operación autónoma	Energizar sin computadora utilizando powerbank o cargador de 5 V	Cumple, no tuvo reinicios después de 10 min
Latencia de comando desde el teléfono hasta el sistema vía WiFi menor a 200 ms	Cronómetro en la app con 11 muestras	150.4 ms promedio
Aislamiento entre control (5 V) y carga (127 V)	Optoacopladores del módulo	Cumple por diseño del dimmer

2 Hardware y plataforma

2.1 Principio de operación

La red eléctrica es $v(t) = V_p sen(2\pi ft)$ con $V_p = \sqrt{2}V_{RMS} = (\sqrt{2})(127) \approx 179.6\,V$ y $f = 60\,Hz$ en hogares y pequeños comercios en México, el semiciclo dura 8.33 ms y hay dos cruces por cero por cada ciclo (120 por segundo). El TRIAC conduce cuando recibe un pulso en su compuerta y se apaga solo en el siguiente cruce por cero. Si se retrasa el disparo un ángulo $\alpha = \omega t$ respecto al cruce, la carga recibe solo la parte final del semiciclo (Fig. 1). Para una carga resistiva, esta relación es $\frac{P}{Pmax} = 1 - \frac{\alpha}{\pi} + \frac{sen(2\alpha)}{2\pi}$, que no es lineal: con $\alpha = 90°$ se entrega el 50 %. Por eso se sincroniza el disparo con la fase de la red en lugar de usar PWM, que varía el ciclo de trabajo de una señal continua y no permite recortar una onda alterna en el punto deseado.

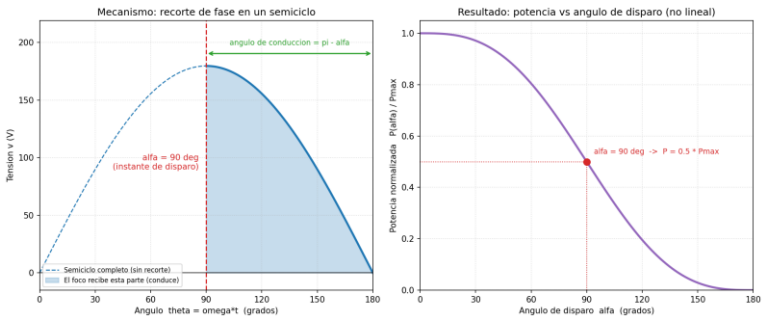


Figura 1. Recorte de fase en un semiciclo (izquierda) y potencia normalizada contra ángulo de disparo (derecha).

2.2 Arquitectura y conexiones

El dispositivo Android se comunica por WiFi con el ESP32-S3 de la placa, que enlaza por una UART interna con el MCU RA4M1. Este lee el cruce por cero en D2 y dispara el TRIAC por D3 (Fig. 2).

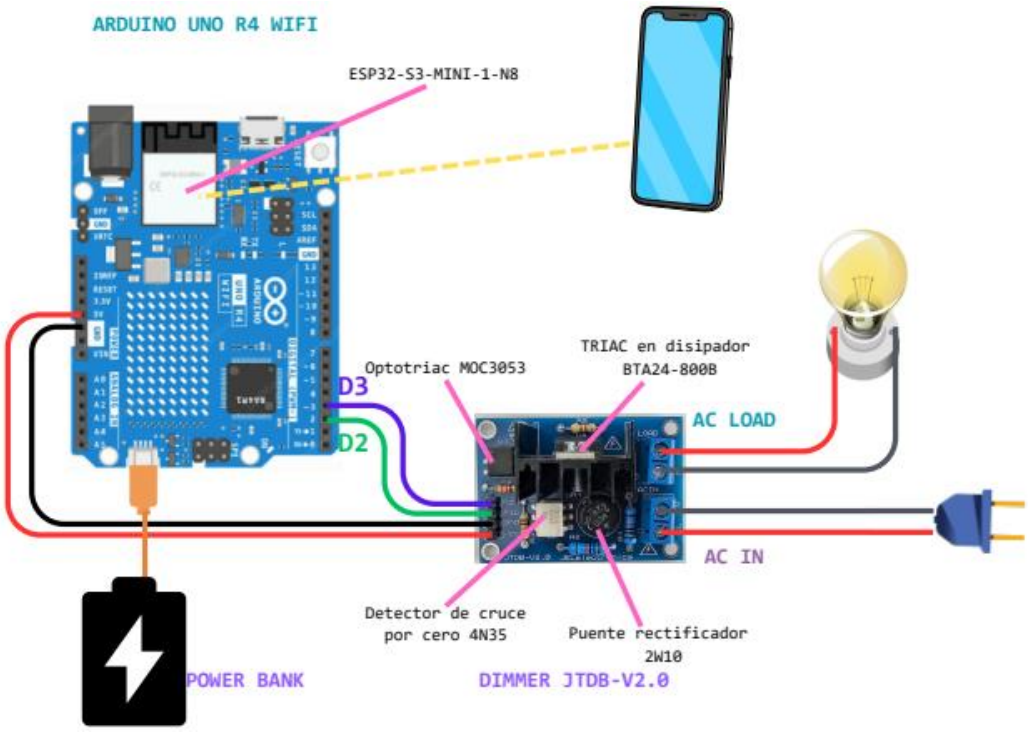


Figura 2. Diagrama de conexiones: Arduino UNO R4 WiFi, módulo dimmer, lámpara, powerbank y teléfono.

Módulo dimmer	Arduino	Función
VCC	5V	Alimentación del dimmer
GND	GND	Tierra
PZC	D2	Cruce por cero (interrupción por flanco de subida)
PD	D3	Disparo del TRIAC (salida, nivel alto)

2.3 Componentes

Componente	Función
Arduino UNO R4 WiFi	RA4M1 (Cortex-M4, 48 MHz, 256 KB flash, 32 KB SRAM): control de fase y servidor HTTP. ESP32-S3: módulo WiFi.
Módulo dimmer JTDB-V2.0 de JElectronica	Etapa de potencia comercial de AC con detector de cruce por cero (2W10 + 4N35) y disparo (MOC3053 + TRIAC BTA24-800B)
Lámpara incandescente	100 W, 110–130 V; carga resistiva
Powerbank o cargador	Alimentación de 5 V para operación autónoma

2.4 Justificación de la plataforma

Se eligió el UNO R4 WiFi por criterios técnicos

1. **Aislamiento entre el manejo de la red y control de la potencia:** El módulo WiFi y TCP/IP corre en el ESP32-S3 mientras que control de fase corre en el microcontrolador RA4M1, por lo que el tráfico de red no compite con el disparo del TRIAC por los recursos del mismo procesador.

2. **Respuesta en tiempo:** El disparo del TRIAC debe ocurrir en un instante preciso de cada semiciclo (cada 8.33 ms), con una resolución de 50 µs; si se retrasa, cambia la potencia entregada y la lámpara parpadea. La RA4M1 cumple esto porque la interrupción externa (cruce por cero) y el temporizador de ticks de 50 µs se ejecutan directamente en hardware, sin otro programa que compita por el procesador, de modo que el tiempo entre el evento y la respuesta es corto. Por otro lado, una Raspberry Pi con Linux ejecuta el programa de control en espacio de usuario, donde el planificador del sistema operativo reparte el procesador entre muchos procesos y puede retrasar la ejecución varios milisegundos; no existe garantía de cumplir un plazo de 50 µs. Cabe aclarar que lo anterior es meramente teórico ya que no fue corroborada en este proyecto por falta de presupuesto.
3. **Arranque y autonomía:** Dado que el programa reside en flash, inicia en segundos al alimentarse con 5 V sin apagado repentino, no requiere conexión hacia la computadora para ejecutar alguna subrutina de arranque o algo parecido.
4. **Recursos suficientes:** El firmware solo ocupa 24% de la memoria flash y 23 % de la memoria dinámica.
5. **WiFi en modo AP integrado, código y hardware abierto:** Al tener un módulo ESP32-S3 integrado es más fácil realizar proyectos que involucren conectividad inalámbrica. Arduino, al ser open-source, existe una enorme comunidad que comparte tutoriales, librerías y códigos listos para usar.

Se tiene en consideración que plataformas como ESP32 funcionan para este proyecto. En Arduino, el control de fase en RA4M1 es bare-metal y simple, mientras que en ESP32 se programa sobre FreeRTOS, con prioridades y núcleos que hay que gestionar. Siendo más flexible pero más complejo.

2.5 Periféricos utilizados

GPIO con interrupción externa (D2, cruce por cero, flanco de subida con pull-up), GPIO de salida (D3, compuerta), temporizador GPT *(General PWM Timer)* en modo periódico a 20 kHz (T = 50 µs con 166 ticks por semiciclo), UART interna entre RA4M1 y ESP32-S3 (librería WiFiS3); USB-C solo para alimentación.

3 Firmware

El sistema **no emplea un sistema operativo**: es firmware *bare-metal* con arquitectura de primer y segundo plano. El código reside en flash y, al energizar, `setup()` inicializa el dimmer, crea el punto de acceso y abre el servidor; si un paso falla, la carga se apaga y el LED integrado parpadea 1, 2 o 3 veces según el error. Las dos actividades concurrentes son el control de fase y la atención de peticiones HTTP. Se separan de la siguiente manera:

Contexto	Disparador	Frecuencia	Función
onZeroCross (ISR)	Flanco de subida en D2	120 Hz	Reinicia la cuenta y baja la compuerta
onTick (ISR)	Temporizador GPT	20 kHz	Incrementa la cuenta; sube la compuerta al llegar al umbral y la baja MARGEN ticks antes del siguiente cruce
loop() (segundo plano)	Ejecución continua	Sin período	Atiende peticiones HTTP, valida y actualiza el umbral
ESP32-S3	Independiente	Asíncrono	Pila WiFi y TCP/IP

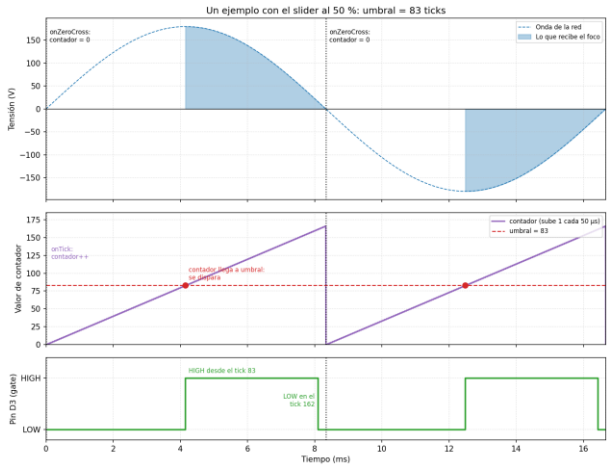


Figura 3. Línea de tiempo con el slider al 50 % (umbral = 83 ticks): cruce por cero, contador y pin de compuerta.

1) **Comunicación y sincronización.** El único dato que viaja del segundo plano a las ISR es `umbral` (variable `volatile`), escrito únicamente por `loop()` y leído por las ISR (*Interrupt Service Routine*); al ser una escritura de 32 bits alineada es atómica en Cortex-M4, por lo que no se requiere exclusión mutua. Las ISR no bloquean ni usan delay, de modo que atender una petición HTTP no altera el disparo.
2) **Planificación.** Las ISR actúan como tareas periódicas de prioridad fija con expulsión sobre `loop()`. La utilización de la ISR de 20 kHz es $U = \frac{C}{T}$, con T = 50 µs; el tiempo de ejecución C no ha sido medido, por lo que no se reporta un valor.
3) **Limitaciones.**
   a) El contador se modifica en ambas ISR; si hubiera expulsión entre ellas, un reinicio podría perderse un semiciclo, y la mitigación es que solo `onTick` lo escriba.
   b) El servidor atiende un cliente a la vez, con tiempo máximo de espera de 500 ms.
   c) Un RTOS permitiría tareas con prioridades, pero se descartó por complejidad frente a un plazo de 50 µs como se mencionó anteriormente.

4 Software y programación

El firmware está escrito en C++ (cadena de herramientas nativa de la plataforma) y modularizado con una responsabilidad por archivo: `dimmer` (control de fase), `wifi_ap` (red), `http_api` (protocolo y validación), `config.h` (constantes) y `dimmer_wifi.ino` (arranque). Las ISR acceden a los periféricos mediante `attachInterrupt` y `FspTimer`. La calibración mapea el control 1–100 % al rango útil del filamento (11–90 % de conducción): $umbral = (100 - real)\frac{166}{100}$, con valores de 147 (1 %), 83 (50 %) y 16 (100 %); 0 % deja el umbral fuera de rango (176) y la carga apagada.

Error	Detección	Respuesta
Temporizador no disponible	Retorno de la inicialización	Carga apagada; LED con 1 parpadeo
Módulo WiFi ausente	`WiFi.status()`	LED con 2 parpadeos
Fallo al crear el AP	Retorno de `beginAP`	LED con 3 parpadeos
Parámetro power inválido (fuera de 0–100, no numérico)	Validación en `http_api`	HTTP 400
Ruta inexistente	Enrutamiento	HTTP 404
Cliente sin datos	Timeout de 500 ms	Se libera el servidor

1) **Depuración.**
   a) El sketch de diagnóstico confirmó 120 pulsos por segundo en D2 y la lógica del nivel de disparo.

b) El sistema saturaba hacia 30 %; se descartó un desajuste del temporizador al medir 166 ticks por semiciclo y se descartó la polaridad de la compuerta; el comportamiento se corrigió al bajar la compuerta MARGEN = 4 ticks antes del cruce por cero y al incrementar el contador de forma incondicional. La causa física no se verificó con osciloscopio.
c) Calibración del rango útil observando el filamento: sin brillo apreciable por debajo de 11 % y sin cambio por encima de 90 %.

2) Aplicación Android.

Está desarrollada en Kotlin con *Minimum SDK: API 26 (Android 8.0),* la petición `GET /set?power=N` se ejecuta en un hilo en segundo plano, porque Android prohíbe operaciones de red en el hilo principal (`NetworkOnMainThreadException`), y se envía al soltar el slider para no saturar al microcontrolador. Se habilita `usesCleartextTraffic` por usar HTTP sin cifrado en una red local aislada.

5 Integración y resultados

El sistema completo (teléfono, AP, firmware, módulo, lámpara) se probó de extremo a extremo con alimentación por powerbank, sin computadora conectada. El video de demostración está en la portada.

Métrica	Valor	Herramienta
Cruces por cero detectados	120 por segundo	Sketch de diagnóstico
Memoria de programa	64 460 B (24 % de 262 144 B)	Compilador de Arduino
Memoria dinámica	7 652 B (23 % de 32 768 B)	Compilador de Arduino
Memoria de la app	120 MB (aprox), estable	Profiler de Android Studio
CPU de la app	1 % en reposo, picos al enviar información	Profiler de Android Studio
Latencia de comando	150.4 ms promedio mín. 129, máx. 178 ms (n = 11)	Cronómetro en la app
Rango útil de control	11 % a 90 %	Observación del filamento
Autonomía	Sin reinicio tras más de 10 min sin mover el slider	Prueba con powerbank

La latencia cumple en ser menor a 200 ms y se percibe inmediata en iluminación incluye abrir una conexión HTTP por petición y el paso por la UART interna. Sin la primera muestra (178 ms) el promedio es 147.6 ms. La memoria del firmware deja amplio margen. La diferencia de escala entre el firmware (7.6 KB de RAM aprox.) y la aplicación (120 MB aprox.) ilustra la distinción entre un sistema embebido y uno de propósito general.

6 Referencias

- Arduino SRL. (2026). Arduino UNO R4 WiFi: User manual (SKU ABX00087, rev. 9) [Hoja de datos]. https://docs.arduino.cc/resources/datasheets/ABX00087-datasheet.pdf
- Matamoros, M. (2020). Apuntes de Fundamentos de Sistemas Embebidos [Apuntes de clase]. Fundamentos de Sistemas Embebidos, Facultad de Ingeniería, UNAM.
- Sandoval Rodríguez, R. U. (2026) Apuntes de Fundamentos de Sistemas Embebidos [Apuntes de clase]. Fundamentos de Sistemas Embebidos, Facultad de Ingeniería, UNAM.
- (s. f.). Módulo dimmer de luz AC control potencia 1 canal [Ficha de producto]. Mercado Libre México. https://www.mercadolibre.com.mx/modulo-dimmer-de-luz-ac-control-potencia-1-canal/up/MLMU721185363
- Javier Arévalo (2021). [Dimmer AC con arduino controlado remotamente || Proyectos con Arduino] [Video]. YouTube. https://www.youtube.com/watch?v=87rZo5v8u78

7 Declaratoria de uso de IA

Herramienta y versión: Claude Sonnet 5.5 (Anthropic), con esfuerzo de razonamiento medio, usado en conversación (claude.ai / Claude Code) durante el desarrollo del proyecto y de este reporte. Se utilizó solo esta herramienta.

Parte del proyecto	Uso de la IA	Verificación y aportación propia
Selección del módulo dimmer y de la etapa de potencia	Orientación sobre alternativas (módulo comercial, MOC3053) y explicación de la función de sus componentes	Adquirí el módulo JTDB-V2.0 y verifiqué sus componentes y pines (PD, PZC, GND, VCC) en la tarjeta física.
Firmware: ISR de cruce por cero, temporizador a 20 kHz, calibración y servidor HTTP	Generación y corrección iterativa de código; explicación del funcionamiento.	Compilé y probé con la lámpara; medí 120 pulsos por segundo y 166 ticks por semiciclo; observé la saturación al 30 % y comprobé la corrección con la variable `MARGEN` determiné el rango útil de 11-90 % observando el filamento.
Modularización y manejo de errores del firmware	Reestructuración del código en módulos y propuesta de códigos de error y respuestas HTTP 400/404	Compilé y probé el firmware modularizado en la placa. Pedí explicación del funcionamiento del código modularizado.
Aplicación Android (Kotlin)	Generación del código de la app (slider, petición HTTP en hilo secundario) y del cronómetro de latencia	Instalé y probé la app en un teléfono físico; capturé las 11 muestras de latencia y revisé los datos de rendimiento en Android Profiler.
Análisis de resultados	Cálculo de media y desviación estándar de las 11 muestras	Las mediciones son propias; la media (150.4 ms) fue recalculada y corregida
Reporte y figuras	Esqueleto del reporte, redacción de apartado 1-5, scripts de las figuras 1 y 2.	Revisé y modifiqué el texto, corregí afirmaciones, agregué la aclaración sobre lo no verificado y realicé el diagrama de conexiones en Canva.

Archivos y secciones asistidos por IA: dimmer_wifi.ino, config.h, dimmer.h/.cpp, wifi_ap.h/.cpp, http_api.h/.cpp, el código de la app Android, los scripts de las figuras y el borrador de las secciones 1 a 5 de este reporte.

Aportación propia: montaje y cableado del hardware, pruebas con la lámpara, calibración del rango útil, mediciones de latencia, uso de memoria y rendimiento de la app, decisiones de diseño (modo AP, app nativa en Kotlin, alimentación autónoma por powerbank) y revisión final del reporte. Las limitaciones no verificadas (causa física de MARGEN, tiempo de ejecución de las ISR) se declaran en el texto.

Declaro que este proyecto es de mi autoría, que la información de esta declaratoria es veraz y que puedo explicar y modificar cualquier parte del código y del reporte.



Jiménez Chávez Daniel Orlando

28 de octubre de 2026